Name: Shailendra Suman
Regd. No: 2141019394

System Monitor Tool

Objective: Create a system monitor tool in C++ that displays real-time information about system processes, memory usage, and CPU load, similar to the 'top' command.

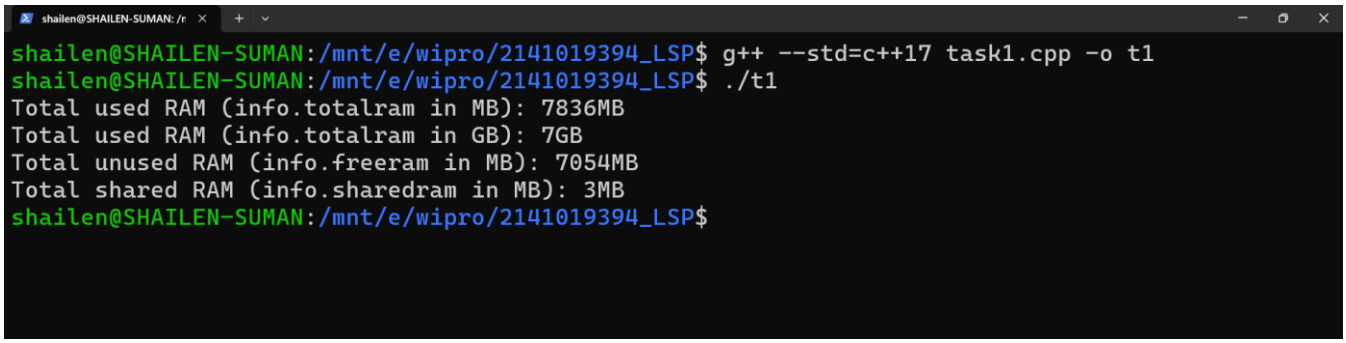
Day-wise Tasks:

Day 1: Design UI layout and gather system data using system calls.

Code:

```
#include <iostream>

using namespace std;

#include <sys/sysinfo.h>

void displayMemoryInfo(){

	struct sysinfo info;

	if (sysinfo(&info)==0){

		cout<< "Total used RAM (info.totalram in MB): "<<info.totalram/(1024*1024)<<"MB\n";

		cout<< "Total used RAM (info.totalram in GB): "<<(info.totalram/(1024*1024))/1024<<"GB\n";

		cout<< "Total unused RAM (info.freeram in MB): "<<info.freeram/(1024*1024)<<"MB\n";

		cout<< "Total shared RAM (info.sharedram in MB): "<<info.sharedram/(1024*1024)<<"MB\n";

	}

}

int main(){

	displayMemoryInfo();

	return 0;

}
```

Output:



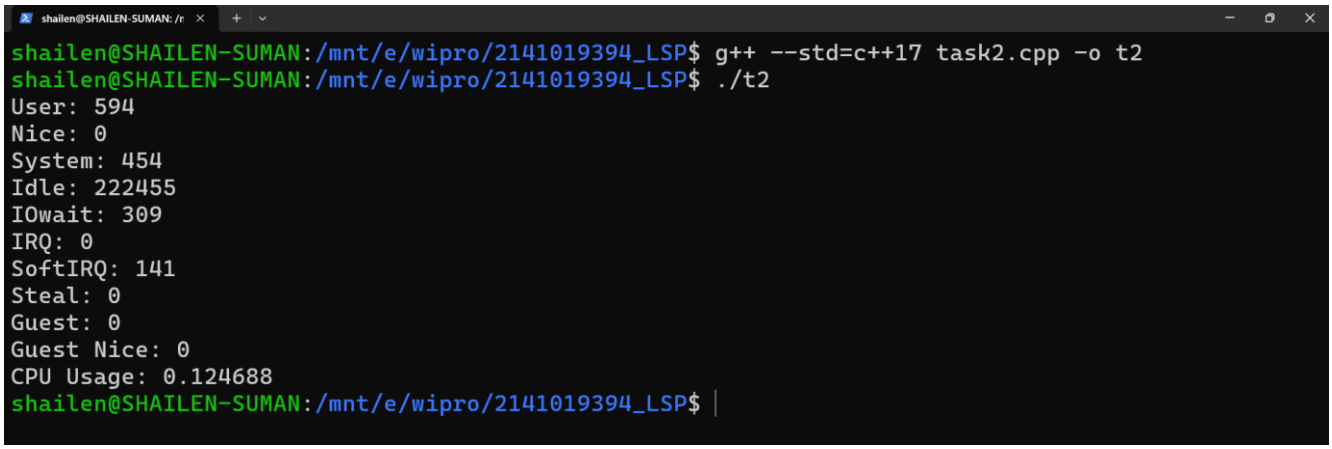
Day 2: Display process list with CPU and memory usage.

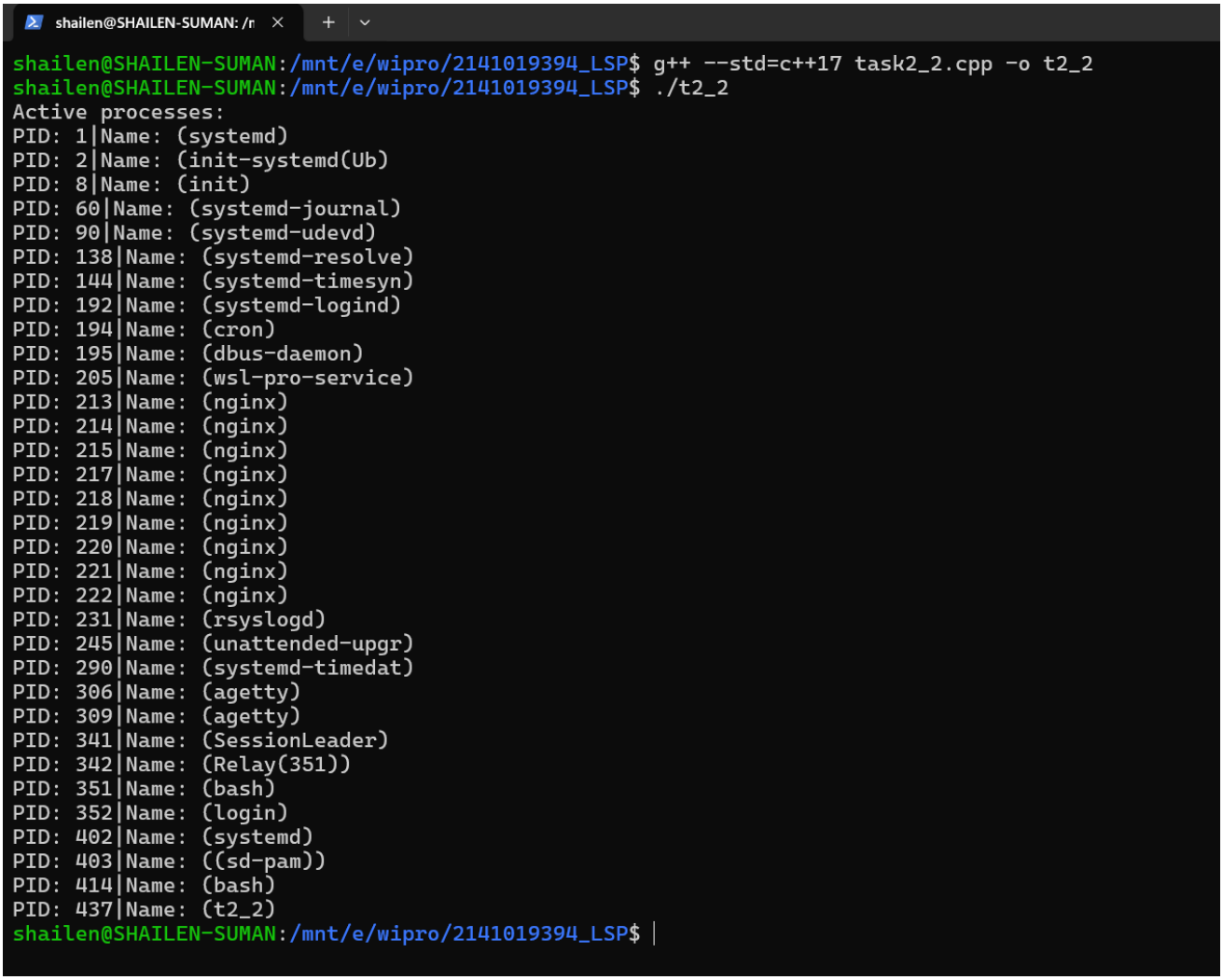
Code:

```
#include <iostream>

#include <fstream>

#include <sstream>

#include <thread>

#include <chrono>

using namespace std;

struct CPUData{

        long user, nice, system, idle, iowait, irq, softirq, steal, guest, guest_nice;

};

CPUData getCPUData(){

        ifstream file("/proc/stat");

        string line;

        CPUData cpu={};

        if (file.is_open()){

                getline(file,line);

                istringstream ss(line);

                string cpuLabel;

                ss >> cpuLabel >> cpu.user >> cpu.nice >> cpu.system >> cpu.idle >> cpu.iowait >>
cpu.irq >> cpu.softirq >> cpu.steal >> cpu.guest >> cpu.guest_nice;
```

```
        }
        return cpu;
}
double calculateCPUUsage (CPUData prev, CPUData current){
        long prevIdle= prev.idle + prev.iowait;
        long currIdle= current.idle + current.iowait;
        long prevTotal = prev.user + prev.nice + prev.system + prev.idle + prev.iowait + prev.irq +
prev.softirq + prev.steal;
        long currTotal = current.user + current.nice + current.system + current.idle + current.iowait +
current.irq + current.softirq + current.steal;
        long totalDiff = currTotal - prevTotal;
        long idleDiff = currIdle - prevIdle;
        return ((totalDiff - idleDiff) * 100.0) / totalDiff;
}
int main(){
        CPUData cpu= getCPUData();
        cout << "User: " << cpu.user << endl;
   cout << "Nice: " << cpu.nice << endl;
   cout << "System: " << cpu.system << endl;
   cout << "Idle: " << cpu.idle << endl;
   cout << "IOwait: " << cpu.iowait << endl;
   cout << "IRQ: " << cpu.irq << endl;
   cout << "SoftIRQ: " << cpu.softirq << endl;
   cout << "Steal: " << cpu.steal << endl;
   cout << "Guest: " << cpu.guest << endl;
   cout << "Guest Nice: " << cpu.guest_nice << endl;
   CPUData prevData = getCPUData();
   this_thread::sleep_for(chrono::seconds(1));
   CPUData currData = getCPUData();
   double cpuUsage= calculateCPUUsage (prevData, currData);
   cout << "CPU Usage: "<<cpuUsage<<endl;
        return 0;
}
```

Output:


shailen@SHAILEN-SUMAN: /r
shailen@SHAILEN-SUMAN:/mnt/e/wipro/2141019394_LSP$ g++ --std=c++17 task2.cpp -o t2
shailen@SHAILEN-SUMAN:/mnt/e/wipro/2141019394_LSP$ ./t2
User: 594
Nice: 0
System: 454
Idle: 222455
IOwait: 309
IRQ: 0
SoftIRQ: 141
Steal: 0
Guest: 0
Guest Nice: 0
CPU Usage: 0.124688
shailen@SHAILEN-SUMAN:/mnt/e/wipro/2141019394_LSP$


shailen@SHAILEN-SUMAN: /r
shailen@SHAILEN-SUMAN:/mnt/e/wipro/2141019394_LSP$ g++ --std=c++17 task2_2.cpp -o t2_2
shailen@SHAILEN-SUMAN:/mnt/e/wipro/2141019394_LSP$ ./t2_2
Active processes:
PID: 1|Name: (systemd)
PID: 2|Name: (init-systemd(Ub)
PID: 8|Name: (init)
PID: 60|Name: (systemd-journal)
PID: 90|Name: (systemd-udevd)
PID: 138|Name: (systemd-resolve)
PID: 144|Name: (systemd-timesyn)
PID: 192|Name: (systemd-logind)
PID: 194|Name: (cron)
PID: 195|Name: (dbus-daemon)
PID: 205|Name: (wsl-pro-service)
PID: 213|Name: (nginx)
PID: 214|Name: (nginx)
PID: 215|Name: (nginx)
PID: 217|Name: (nginx)
PID: 218|Name: (nginx)
PID: 219|Name: (nginx)
PID: 220|Name: (nginx)
PID: 221|Name: (nginx)
PID: 222|Name: (nginx)
PID: 231|Name: (rsyslogd)
PID: 245|Name: (unattended-upgr)
PID: 290|Name: (systemd-timedat)
PID: 306|Name: (agetty)
PID: 309|Name: (agetty)
PID: 341|Name: (SessionLeader)
PID: 342|Name: (Relay(351))
PID: 351|Name: (bash)
PID: 352|Name: (login)
PID: 402|Name: (systemd)
PID: 403|Name: ((sd-pam))
PID: 414|Name: (bash)
PID: 437|Name: (t2_2)
shailen@SHAILEN-SUMAN:/mnt/e/wipro/2141019394_LSP$

Day 3: Implement process sorting by CPU and memory usage.

Code:

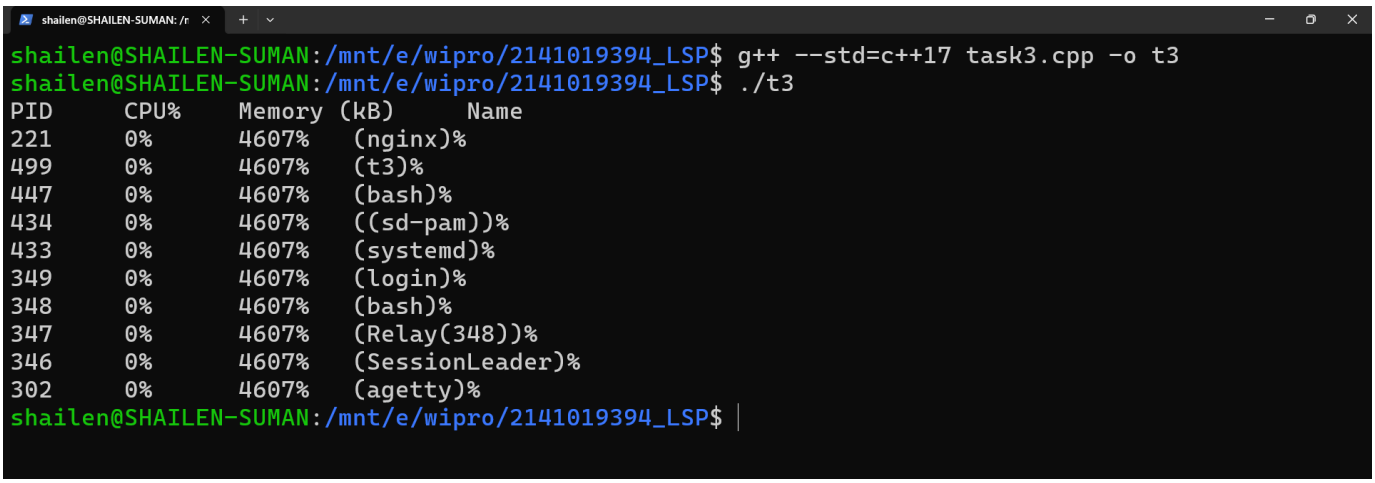
```
#include <iostream>
#include <filesystem>
#include <fstream>
#include <sstream>
#include <algorithm>
#include <vector>
#include <dirent.h>
#include <unistd.h>
#include <csignal>
using namespace std;
namespace fs=std::filesystem;
struct ProcessInfo{
        int pid;
        string name;
        double cpuUsage;
        long memoryUsage;
};
string readFileValue(const string &path){
        ifstream file(path);
        string value;
        if (file.is_open()){
                getline(file,value);
        }
        return value;
}
double getSystemUptime(){
        ifstream file("/proc/uptime");
        double uptime;
```

```
        if (file.is_open()){
                file >> uptime;
        }
        return uptime;
}
ProcessInfo getProcessInfo(int pid,double  systemUptime){
        ProcessInfo pinfo;
   pinfo.pid = pid;
        ifstream file("/proc/" + to_string(pid)  + "/stat");
        string line;
        long utime=0, stime=0, starttime=0;
        if(file.is_open()){
                getline(file,line);
                istringstream  ss(line);
                string token;
                int count=0;

                while(ss >> token){
                        count++;
                        if(count==2) pinfo.name=token;
                        else if(count==14) utime=stol(token);
                        else if(count==15) stime=stol(token);
                        else if(count==22) starttime=stol(token);
                }
        }

        ifstream memFile("/proc/"  + to_string(pid)  + "/status");
        if (memFile.is_open()){
                string key, value,unit;
                while(memFile  >> key >> value >> unit){
                        if (key=="VmRSS"){
                                pinfo.memoryUsage=stol(value);
                                break;
```

```
                }
            }
        }
        long total_time = utime + stime;
        double seconds = systemUptime - (starttime / sysconf(_SC_CLK_TCK));
        pinfo.cpuUsage = (total_time / sysconf(_SC_CLK_TCK)) / seconds * 100;
        return pinfo;


}

vector <ProcessInfo> getAllProcesses(){
        vector <ProcessInfo> processes;
        double systemUptime= getSystemUptime();
        for (const auto &entry : fs::directory_iterator("/proc")){
            if (entry.is_directory()){
                string filename=entry.path().filename().string();
                if (all_of(filename.begin(),filename.end(),  ::isdigit)){
                    int pid=stoi(filename);
                    processes.push_back(getProcessInfo(pid,systemUptime));
                }
            }
        }
        return processes;
}

void sortProcesses(vector<ProcessInfo> &processes, bool sortByCpu){
        if(sortByCpu){
            sort(processes.begin(),processes.end(),[](const  ProcessInfo &a, const ProcessInfo
&b)
            {
                return a.cpuUsage > b.cpuUsage;
            });
        }
```

```
        else{
                sort(processes.begin(),processes.end(),[](const  ProcessInfo &a, const ProcessInfo
&b)
                {
                        return a.memoryUsage > b.memoryUsage;
                });
        }
}



int main(){
        vector <ProcessInfo> processes = getAllProcesses();
        sortProcesses(processes,true);
        cout<<"PID\tCPU%\tMemory  (kB)\tName\n";
        for (size_t i=0;i<min(processes.size(),size_t(10));i++){

        cout<<processes[i].pid<<"\t"<<processes[i].cpuUsage<<"%\t"<<processes[i].memoryUsage<
<"%\t"<<processes[i].name<<"%\n";
        }
        return 0;
}
```

Output:



Day 4: Add functionality to kill processes.

Code:

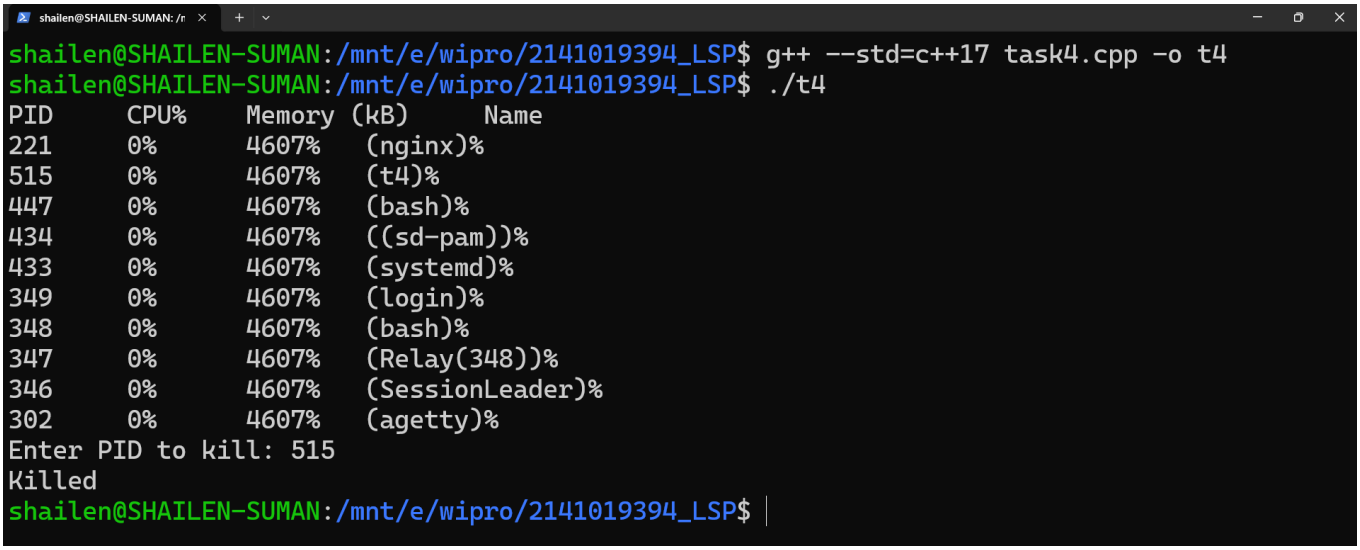
```
#include <iostream>
#include <filesystem>
#include <fstream>
#include <sstream>
#include <algorithm>
#include <vector>
#include <dirent.h>
#include <unistd.h>
#include <csignal>
using namespace std;
namespace fs=std::filesystem;
struct ProcessInfo{
        int pid;
        string name;
        double cpuUsage;
        long memoryUsage;
};
string readFileValue(const string &path){
        ifstream file(path);
        string value;
        if (file.is_open()){
                getline(file,value);
        }
        return value;
}
double getSystemUptime(){
        ifstream file("/proc/uptime");
        double uptime;
```

```
        if (file.is_open()){
                file >> uptime;
        }
        return uptime;
}
ProcessInfo getProcessInfo(int pid,double  systemUptime){
        ProcessInfo pinfo;
   pinfo.pid = pid;
        ifstream file("/proc/" + to_string(pid)  + "/stat");
        string line;
        long utime=0, stime=0, starttime=0;
        if(file.is_open()){
                getline(file,line);
                istringstream  ss(line);
                string token;
                int count=0;

                while(ss >> token){
                        count++;
                        if(count==2) pinfo.name=token;
                        else if(count==14) utime=stol(token);
                        else if(count==15) stime=stol(token);
                        else if(count==22) starttime=stol(token);
                }
        }

        ifstream memFile("/proc/"  + to_string(pid)  + "/status");
        if (memFile.is_open()){
                string key, value,unit;
                while(memFile  >> key >> value >> unit){
                        if (key=="VmRSS"){
                                pinfo.memoryUsage=stol(value);
                                break;
```

```
                }
            }
        }
        long total_time = utime + stime;
        double seconds = systemUptime - (starttime / sysconf(_SC_CLK_TCK));
        pinfo.cpuUsage = (total_time / sysconf(_SC_CLK_TCK)) / seconds * 100;
        return pinfo;


}

vector <ProcessInfo> getAllProcesses(){
        vector <ProcessInfo> processes;
        double systemUptime= getSystemUptime();
        for (const auto &entry : fs::directory_iterator("/proc")){
            if (entry.is_directory()){
                string filename=entry.path().filename().string();
                if (all_of(filename.begin(),filename.end(),  ::isdigit)){
                    int pid=stoi(filename);
                    processes.push_back(getProcessInfo(pid,systemUptime));
                }
            }
        }
        return processes;
}

void sortProcesses(vector<ProcessInfo> &processes, bool sortByCpu){
        if(sortByCpu){
            sort(processes.begin(),processes.end(),[](const  ProcessInfo &a, const ProcessInfo
&b)
            {
                return a.cpuUsage > b.cpuUsage;
            });
        }
```

```
        else{
                sort(processes.begin(),processes.end(),[](const ProcessInfo &a, const ProcessInfo
&b)
                {
                        return a.memoryUsage > b.memoryUsage;
                });
        }
}



int main(){
        vector <ProcessInfo> processes = getAllProcesses();
        sortProcesses(processes,true);
        cout<<"PID\tCPU%\tMemory (kB)\tName\n";
        for (size_t i=0;i<min(processes.size(),size_t(10));i++){

        cout<<processes[i].pid<<"\t"<<processes[i].cpuUsage<<"%\t"<<processes[i].memoryUsage<
<"%\t"<<processes[i].name<<"%\n";
        }
        int targetPid;
        cout<<"Enter PID to kill: ";
        cin>>targetPid;
        if(targetPid > 0){
                if (kill(targetPid, SIGKILL) == 0){
                        cout<<"Process "<< targetPid << " terminated successfully\n";
                }
                else{
                        perror("Failed to kill the process");
                }
        }
        return 0;
}
```

Output:



Day 5: Implement real-time update feature to refresh data every few seconds.

Code:

```
#include <iostream>
#include <filesystem>
#include <fstream>
#include <sstream>
#include <algorithm>
#include <vector>
#include <dirent.h>
#include <unistd.h>
#include <csignal>
#include <thread>
#include <chrono>
using namespace std;
namespace fs=std::filesystem;
struct ProcessInfo{
```

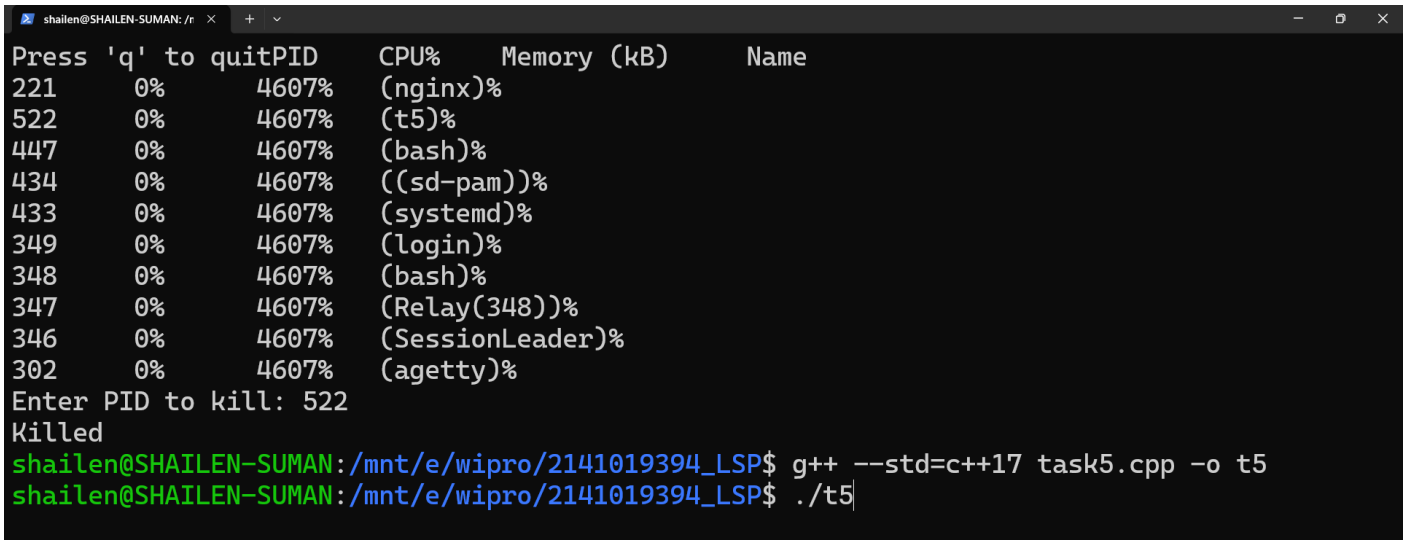
```
        int pid;
        string name;
        double cpuUsage;
        long memoryUsage;
};
string readFileValue(const string &path){
        ifstream file(path);
        string value;
        if (file.is_open()){
                getline(file,value);
        }
        return value;
}
double getSystemUptime(){
        ifstream file("/proc/uptime");
        double uptime;
        if (file.is_open()){
                file >> uptime;
        }
        return uptime;
}
ProcessInfo getProcessInfo(int pid,double systemUptime){
        ProcessInfo pinfo;
   pinfo.pid = pid;
        ifstream file("/proc/" + to_string(pid) + "/stat");
        string line;
        long utime=0, stime=0, starttime=0;
        if(file.is_open()){
                getline(file,line);
                istringstream ss(line);
                string token;
                int count=0;
```

```
            while(ss >> token){
                count++;
                if(count==2) pinfo.name=token;
                else if(count==14) utime=stol(token);
                else if(count==15) stime=stol(token);
                else if(count==22) starttime=stol(token);
            }
        }

        ifstream memFile("/proc/"  + to_string(pid)  + "/status");
        if (memFile.is_open()){
            string key, value,unit;
            while(memFile  >> key >> value >> unit){
                if (key=="VmRSS"){
                    pinfo.memoryUsage=stol(value);
                    break;
                }
            }
        }
        long total_time = utime + stime;
        double seconds = systemUptime - (starttime / sysconf(_SC_CLK_TCK));
        pinfo.cpuUsage = (total_time / sysconf(_SC_CLK_TCK)) / seconds * 100;
        return pinfo;


}

vector <ProcessInfo> getAllProcesses(){
        vector <ProcessInfo> processes;
        double systemUptime= getSystemUptime();
        for (const auto &entry : fs::directory_iterator("/proc")){
            if (entry.is_directory()){
                string filename=entry.path().filename().string();
```

```
                        if (all_of(filename.begin(),filename.end(),  ::isdigit)){
                                int pid=stoi(filename);
                                processes.push_back(getProcessInfo(pid,systemUptime));
                        }
                }
        }
        return processes;
}

void sortProcesses(vector<ProcessInfo> &processes, bool sortByCpu){
        if(sortByCpu){
                sort(processes.begin(),processes.end(),[](const  ProcessInfo &a, const ProcessInfo
&b)
                {
                        return a.cpuUsage > b.cpuUsage;
                });
        }
        else{
                sort(processes.begin(),processes.end(),[](const  ProcessInfo &a, const ProcessInfo
&b)
                {
                        return a.memoryUsage > b.memoryUsage;
                });
        }
}



int main(){
        char input;
        while (true){
                system("clear");
                vector <ProcessInfo> processes = getAllProcesses();
                sortProcesses(processes,true);
                cout<<"Press 'q' to quit";
```

```
                cout<<"PID\tCPU%\tMemory (kB)\tName\n";
                for (size_t i=0;i<min(processes.size(),size_t(10));i++){

        cout<<processes[i].pid<<"\t"<<processes[i].cpuUsage<<"%\t"<<processes[i].memoryUsage<
<"%\t"<<processes[i].name<<"%\n";
                }
                int targetPid;
                cout<<"Enter PID to kill: ";
                cin>>targetPid;
                if (targetPid == 0){
                        //continue refresh
                }
                else if(targetPid > 0){
                        if (kill(targetPid, SIGKILL) == 0){
                                cout<<"Process "<< targetPid << " terminated successfully\n";
                        }
                        else{
                                perror("Failed to kill the process");
                        }
                }

                cout << "\nPress 'q' to quit or Enter to refresh: ";
                cin.ignore( );
                input = getchar( );
                if (input == 'q' || input =='Q')
                break;
                this_thread::sleep_for(chrono::seconds(2));
        }
        return 0;
}
```

Output:


shailen@SHAILEN-SUMAN: /r
Press 'q' to quitPID CPU% Memory (kB) Name
221 0% 4607% (nginx)%
522 0% 4607% (t5)%
447 0% 4607% (bash)%
434 0% 4607% ((sd-pam))%
433 0% 4607% (systemd)%
349 0% 4607% (login)%
348 0% 4607% (bash)%
347 0% 4607% (Relay(348))%
346 0% 4607% (SessionLeader)%
302 0% 4607% (agetty)%
Enter PID to kill: 522
Killed
shailen@SHAILEN-SUMAN:/mnt/e/wipro/2141019394_LSP$ g++ --std=c++17 task5.cpp -o t5
shailen@SHAILEN-SUMAN:/mnt/e/wipro/2141019394_LSP$ ./t5